AI & RAG Guide — Vermont Health Platform (HTR)

Audience: AI/ML engineers, backend developers. **Version:** 4.2.0 **Stack:** LlamaIndex · Groq (Llama 3.3-70b) · OpenAI Embeddings · FlashRank · pgvector

---

Table of Contents

---

1. Overview

The HTR AI Analyst is a Retrieval-Augmented Generation (RAG) system that answers questions about U.S. healthcare transformation by grounding responses in the platform's curated knowledge base. It combines:

- **Hybrid retrieval** — BM25 keyword search merged with dense vector similarity via Reciprocal Rank Fusion (RRF)
- **Cross-encoder re-ranking** — FlashRank (`ms-marco-MiniLM-L-12-v2`) re-scores retrieved chunks before passing to the LLM
- **Role-based LLM routing** — Subscriber/Student users get Groq Llama 3.3-70b; Professional/Advisory users get Claude Sonnet 4.6 for higher-quality responses
- **Sentence window expansion** — Retrieved chunks are expanded ±3 sentences to provide richer context
- **Streaming responses** — All chat responses stream token-by-token back to the browser

---

2. Architecture

```
User query (browser)
  │
  ▼
POST /api/chat (Next.js API route)
  │  Validates schema (zod), checks auth cookie
  │
  ▼
POST /api/chat (FastAPI — Railway)
  │  1. Verify JWT (Supabase JWT secret, HS256)
  │  2. Decode user ID + role from claims
  │  3. Select LLM model based on role
  │
  ▼
Embed query
  │  OpenAI text-embedding-3-small → 768-dim vector
  │
  ▼
Hybrid search (Supabase RPC: hybrid_search_rag)
  │  BM25 (tsvector) + cosine ANN (HNSW) → top-20 chunks
  │  Merged via Reciprocal Rank Fusion (RRF, k=60)
```

```
  │
  ▼
Sentence window expansion
  │  Each chunk → window of ±3 sentences from source node
  │
  ▼
FlashRank re-ranking
  │  ms-marco-MiniLM-L-12-v2 cross-encoder → top-5 chunks
  │
  ▼
Build prompt
  │  System prompt + context chunks + conversation history
  │
  ▼
LLM (Groq Llama 3.3-70b OR Claude Sonnet 4.6)
  │  Streams tokens
  │
  ▼
Stream response → browser
```

---

3. Document Ingestion Pipeline

Source Documents

The knowledge base is built from two source types:

Source	Content	Format
Sanity CMS	Policy analyses, blog posts, academy modules, case studies, definitions	Portable Text → plain text
Static PDFs	Technical reports, white papers, government documents	PDF → extracted text

Sanity GROQ Queries

The following content types are indexed from Sanity (defined in `backend/services/indexing.py`):

```
SANITY_QUERIES = {
    "policyAnalysis": """*[_type=="policyAnalysis" && defined(slug.current)]{
        _id, title, pillar, summary,
        "bodyText": pt::text(body)
    }""",
    "post": """*[_type=="post" && defined(slug.current)]{
        _id, title,
        "bodyText": pt::text(body)
    }""",
    "academyModule": """*[_type=="academyModule" && defined(slug.current)]{
        _id, title, pillar, summary, learningObjectives,
        "bodyText": pt::text(body)
    }""",
    "caseStudy": """*[_type=="caseStudy" && defined(slug.current)]{
        _id, title, pillar, summary,
        "bodyText": pt::text(body)
    }""",
    "definition": """*[_type=="definition"]{
        _id, term, description, pillars
    }""",
}
```

Chunking Strategy — Sentence Window

Documents are chunked using LlamaIndex's `SentenceWindowNodeParser`:

```
from llama_index.core.node_parser import SentenceWindowNodeParser

parser = SentenceWindowNodeParser.from_defaults(
    window_size=3,          # ±3 sentences around the anchor sentence
    window_metadata_key="window",
    original_text_metadata_key="original_text",
)
```

Each chunk is a single sentence (the anchor). At retrieval time, the ±3 sentence window is expanded to provide richer context. This balances precision at retrieval with contextual richness at generation.

Ingestion Flow

```
# Simplified from backend/services/indexing.py

async def build_index():
    # 1. Fetch Sanity content
    documents = await fetch_sanity_content()

    # 2. Load static PDFs from backend/data/
    pdf_docs = SimpleDirectoryReader(DATA_DIR).load_data()
    documents.extend(pdf_docs)

    # 3. Parse into sentence-window nodes
    parser = SentenceWindowNodeParser.from_defaults(window_size=3)
    nodes = parser.get_nodes_from_documents(documents)

    # 4. Embed + store in pgvector
    if SUPABASE_DB_URL:
        vector_store = PGVectorStore.from_params(
            database_url=SUPABASE_DB_URL,
            table_name="rag_documents",
            embed_dim=768,
        )
        storage_context =
StorageContext.from_defaults(vector_store=vector_store)
    else:
        storage_context = StorageContext.from_defaults()  # local JSON fallback

    index = VectorStoreIndex(nodes, storage_context=storage_context)
    return index
```

Triggering Re-ingestion

Ingestion is triggered automatically by Sanity webhooks when content is published. Manual triggers are available:

```
# Manual trigger via API
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer YOUR_INGEST_SECRET"

# Check status
curl https://your-backend.railway.app/api/ingest/status \
  -H "Authorization: Bearer YOUR_INGEST_SECRET"
```

From the admin dashboard, the "Trigger Ingest" button on the `/admin/ingest` page sends the same request.

---

4. Embedding Model

Setting	Value
Provider	OpenAI
Model	`text-embedding-3-small`
Dimensions	768
Max tokens	8,191
Cost	~$0.02 / million tokens

Configured in `backend/services/llm.py`:

```
from llama_index.embeddings.openai import OpenAIEmbedding

embed_model = OpenAIEmbedding(
    model="text-embedding-3-small",
    dimensions=768,
    api_key=OPENAI_API_KEY,
)
Settings.embed_model = embed_model
```

The same model is used for both ingestion (document embedding) and retrieval (query embedding), ensuring the vector space is consistent.

---

5. Vector Store (pgvector)

Embeddings are stored in Supabase PostgreSQL using the `pgvector` extension. The `rag_documents` table holds chunked text alongside 768-dimensional embedding vectors.

Storage Configuration

```
from llama_index.vector_stores.postgres import PGVectorStore

vector_store = PGVectorStore.from_params(
    database_url=SUPABASE_DB_URL,
    table_name="rag_documents",
    embed_dim=768,
    hnsw_kwargs={
        "hnsw_m": 16,
        "hnsw_ef_construction": 64,
        "hnsw_ef_search": 40,
    }
)
```

HNSW Index

The Hierarchical Navigable Small World (HNSW) index enables sub-millisecond approximate nearest-neighbor (ANN) search:

```
CREATE INDEX idx_rag_embedding
  ON public.rag_documents
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

- `m = 16` — graph branching factor. Higher values increase recall but slow index construction.

- `ef_construction = 64` — search width during build. Higher values improve accuracy.
- `ef_search` — set at query time; controls accuracy vs. latency tradeoff.

Fallback — Local JSON Storage

If `SUPABASE_DB_URL` is not set, the index falls back to LlamaIndex's local JSON vector store in `backend/storage/`. This is suitable for development without a Supabase instance.

---

6. Hybrid Retrieval

The `HybridRetriever` class in `backend/services/retrieval.py` calls the `hybrid_search_rag` Supabase RPC, which combines two retrieval methods:

Dense Retrieval (Vector)

Computes cosine similarity between the query embedding and all stored document embeddings using the HNSW index. Fast (sub-10ms for typical corpus sizes). Good at semantic similarity — finds conceptually related content even when keywords differ.

Sparse Retrieval (BM25)

Uses PostgreSQL's built-in full-text search: `to_tsvector` for indexing, `plainto_tsquery` for query parsing, and `ts_rank` for scoring. Good at exact keyword matching. Required for proper nouns, acronyms, and technical terms (e.g., "FHIR", "AHEAD model", "APM").

Reciprocal Rank Fusion

Both result lists are merged using RRF with `k=60`:

```
RRF score = (dense_weight / (60 + dense_rank)) + (sparse_weight / (60 +
sparse_rank))
```

Default weights: `dense=0.6`, `sparse=0.4`. Adjust in `hybrid_search_rag` SQL function if retrieval quality needs tuning.

Retrieval Parameters

Parameter	Default	Purpose
`top_k`	20	Chunks returned by hybrid search before re-ranking
Final chunks	5	Chunks passed to LLM after re-ranking
`window_size`	3	Sentence window expansion (±3 sentences)

---

7. Re-ranking

After hybrid retrieval returns 20 candidates, FlashRank applies cross-encoder re-ranking to select the top 5 most relevant chunks.

FlashRank Configuration

```
# backend/services/llm.py
from flashrank import Ranker, RerankRequest

_ranker: Ranker | None = None

def get_ranker() -> Ranker | None:
    global _ranker
    if _ranker is None:
```

```
        try:
            _ranker = Ranker(model_name="ms-marco-MiniLM-L-12-v2")
        except Exception as e:
            log.warning(f"FlashRank not available: {e}")
    return _ranker
```

Model: `ms-marco-MiniLM-L-12-v2` — a cross-encoder fine-tuned on MS MARCO passage ranking. It scores (query, passage) pairs directly rather than comparing embeddings, which gives significantly better ranking quality.

The model downloads to `/tmp/flashrank` on first use (~100MB). Subsequent startups reuse the cached model.

Re-ranking Function

```
# backend/services/retrieval.py
def rerank_nodes(query: str, nodes: List[NodeWithScore], top_k: int = 5) ->
List[NodeWithScore]:
    ranker = get_ranker()
    if ranker is None or not nodes:
        return nodes[:top_k]

    passages = [{"id": i, "text": n.node.get_content()} for i, n in
enumerate(nodes)]
    request = RerankRequest(query=query, passages=passages)
    results = ranker.rerank(request)

    reranked = []
    for r in results[:top_k]:
        original = nodes[r["id"]]
        reranked.append(NodeWithScore(node=original.node, score=r["score"]))
    return reranked
```

---

8. LLM Routing

The model used for generation is determined by the user's role. This is resolved in `backend/services/auth.py` when the JWT is decoded.

Role	Model	Characteristics
`free`	Groq Llama 3.1-8b	Fastest, lower quality
`subscriber` / `student`	Groq Llama 3.3-70b	High quality, low latency
`professional` / `advisory`	Claude Sonnet 4.6	Highest quality, longer context
`admin`	Groq Llama 3.3-70b	Same as subscriber

Configuration in `backend/config.py`:

```
MODEL_FREE       = "llama-3.1-8b-instant"    # Groq
MODEL_SUBSCRIBER = "llama-3.3-70b-versatile"  # Groq
MODEL_ADVISORY   = "claude-sonnet-4-6"        # Anthropic
```

Dev Bypass

When `SUPABASE_JWT_SECRET` is not set in the backend environment, all requests are treated as `role="subscriber"` (Llama 3.3-70b). This allows local development without a valid JWT.

---

9. Chat API

Endpoint

```
POST /api/chat
Auth: Bearer JWT (Supabase user token)
Rate: 30/min per IP (slowapi)
```

Request Schema

```python
class ChatRequest(BaseModel):
    message:      str             # max 2000 chars
    history:      list[dict]      # max 100 items, each {role, text}, max 4000 chars/item
    temperature:  float = 0.7     # 0.0–1.0
    systemPrompt: str = ""        # max 800 chars; appended to default system prompt
```

Response

`text/plain` stream. The frontend reads the stream with `ReadableStream` and appends tokens to the UI as they arrive.

System Prompt

Defined in `routers/chat.py`:

```
SYSTEM_PROMPT = """You are the HTR AI Analyst — an expert in U.S. healthcare transformation across policy, economics, technology, clinical innovation, and health equity.

Answer questions using ONLY the provided context. If the context does not contain enough information to fully answer, say so clearly.

Cite your sources when possible. Format responses with clear headings and bullet points where appropriate. Keep responses focused and actionable.
"""
```

Follow-up Suggestions

```
POST /api/suggest
Auth: optional
Rate: 60/min per IP
Body: { "topic": "string" }
Response: { "suggestions": ["string", "string", "string"] }
```

Returns 3 contextual follow-up question suggestions displayed below each AI response in the chat UI.

---

10. Personalized Learning Generation

The Personalized Learning feature (`/academy/personalized-learning`) generates a multi-week structured learning curriculum using a separate LLM call, not the RAG pipeline.

Endpoint

```
POST /api/personalized-learning/generate
Auth: Bearer JWT
```

Request Schema

```python
class LearningPathRequest(BaseModel):
    role:               str
    topics:             list[str]
    difficulty:         Literal["foundational", "intermediate", "advanced"]
    weekly_hours:       str       # e.g. "3–5"
    goals:              str
    format_preferences: list[str] = []
```

Generation Strategy

The backend constructs a detailed system prompt instructing the LLM to generate a JSON curriculum with structured weeks, items, and knowledge checks. The curriculum is returned as structured JSON and stored in `user_learning_paths`.

Text-to-Speech

```
POST /api/personalized-learning/tts
Auth: Bearer JWT
Body: { "text": "...", "voice": "alloy|echo|fable|onyx|nova|shimmer" }
Response: audio/mpeg (streaming MP3)
```

Uses OpenAI TTS API (`tts-1` model). The frontend plays the audio directly in the browser.

---

11. Extending the Pipeline

Adding a New Content Type to the Index

In `backend/services/indexing.py`, add a GROQ query to `SANITY_QUERIES`:

```python
SANITY_QUERIES["myNewType"] = """*[_type=="myNewType" && defined(slug.current)]
{
    _id, title, pillar, summary,
    "bodyText": pt::text(body)
}"""
```

After adding, trigger a re-ingest. The new content type will be chunked and embedded automatically.

Adding a Static Document

Place a PDF or text file in `backend/data/`. The `SimpleDirectoryReader` ingests all files in this directory on each rebuild. Re-ingest to index the new file.

Changing Retrieval Parameters

Adjust `top_k` in the `HybridRetriever` constructor and the final `top_k` in `rerank_nodes()`:

```python
# In routers/chat.py
retriever = HybridRetriever(supabase=sb, top_k=30)  # retrieve more candidates
nodes = rerank_nodes(query, nodes, top_k=8)          # pass more to LLM
```

Changing RRF Weights

Edit the `dense_weight` and `sparse_weight` variables inside the `hybrid_search_rag` SQL function in Supabase. Increase `sparse_weight` if users frequently query by exact terms; increase `dense_weight` for more conceptual/semantic questions.

Adding an Agentic Tool

Define a tool in `backend/services/tools.py` and register it with the ReAct agent in `routers/chat.py`:

```
from llama_index.core.tools import FunctionTool

def query_state_rank(state: str) -> str:
    """Returns the Performance Index rank for a given U.S. state."""
    # fetch from Supabase state_performance_index
    ...

query_state_rank_fn = FunctionTool.from_defaults(fn=query_state_rank)
```

Tools let the agent perform database lookups, calculations, or API calls instead of relying solely on the vector index.

---

12. Monitoring & Debugging

Health Check

```
curl https://your-backend.railway.app/health
```

Returns:

```
{
  "status": "ok",
  "version": "4.2.0",
  "index_ready": true,
  "model_subscriber": "llama-3.3-70b-versatile",
  "embedding_model": "text-embedding-3-small",
  "vector_store": "pgvector",
  "auth_enabled": true,
  "reranker": "flashrank",
  "retrieval": "hybrid_bm25_vector_rrf",
  "chunking": "sentence_window_3"
}
```

Query Log

Every AI Analyst query is logged to `rag_query_log` in Supabase. The admin dashboard at `/admin/ingest` shows recent queries with latency, model used, and whether retrieval returned zero results (`was_zero_result`).

Zero-result queries indicate that no chunks scored above the similarity threshold. Common causes:

- Query is about a topic not in the knowledge base
- Misspelling or unusual terminology (try spelling out acronyms)
- Content published but not yet re-ingested

Debug Logging

The backend logs retrieval details at `INFO` level:

```
2026-03-15 10:23:11 INFO htr-brain: Hybrid search returned 18 nodes
2026-03-15 10:23:11 INFO htr-brain: FlashRank re-ranked to top 5
2026-03-15 10:23:11 INFO htr-brain: Streaming response via llama-3.3-70b-
versatile
2026-03-15 10:23:12 INFO htr-brain: Chat complete — 842ms
```

Set `LOG_LEVEL=DEBUG` in the backend `.env` for verbose retrieval scoring output.

Common Issues

"Index not ready" — The startup build or load failed. Check Railway logs for the error. Common causes: missing `SUPABASE_DB_URL`, pgvector extension not enabled, or `OPENAI_API_KEY` invalid (embedding fails during build).

High latency (>3s) — FlashRank may be downloading its model on first startup (one-time ~100MB download). Check `/tmp/flashrank` for the cached model. If latency persists, check `rag_query_log.latency_ms` to identify whether the bottleneck is embedding, retrieval, re-ranking, or LLM generation.

Poor answer quality — Check `was_zero_result` in the query log. If zero results, add more content to the knowledge base and re-ingest. If results are returned but answers are poor, review the `retrieved_doc_ids` to see what context was passed to the LLM.

JWT validation errors — Verify `SUPABASE_JWT_SECRET` in the backend `.env` exactly matches Supabase → Project Settings → API → JWT Secret. The secret is used to verify HS256 signatures.